What is still missing to make this truly enterprise grade

Prepared for Michael Lyman · Greenway Marijuana · books-38

How I built this list

You asked for this twice, and you asked for it honestly, so let me say up front how I produced it.

Every claim below was checked against the code today, by running searches and probes, not by remembering what I built. Where I say something does not exist, I ran the search and I am quoting the result. Where I say something exists but is not connected, I checked both ends. I have marked each item with what it would take and why it matters, and I have separated the things I can build from the things only you can supply.

I have also included a section of things that are genuinely done, because a gap report that lists only holes gives you a distorted picture of where you stand — and you are, in fact, standing somewhere quite good.

For scale: the system is 1,653 TypeScript files, 201 database migrations, and 8,942 tests across 403 test files, all passing as of this commit.

The one-paragraph summary

The payroll engines are built, correct, and heavily tested. What is missing is mostly assembly: the calculation engine, the timesheet reader, the year-to-date accumulator, the W-4 store, and the garnishment engine all exist and all work, but nothing yet calls them in sequence to produce an actual paycheque for an actual employee. That is one slice of work — the single most valuable slice left — and it is what stands between you and the parallel run you asked about. Behind that sit the annual filing forms (W-2, 941, 940), the financial statements, and roughly 3,575 lines of written guidance that no screen currently displays.

PART ONE — The critical gap: nothing computes a real paycheque yet

1.1 There are two payroll screens, and they are not connected to each other

This is the most important thing in this document, so I want to be precise.

Screen A — `/admin/payroll`. This exists and works. It creates pay runs, saves lines, evaluates guardrails, and generates the NACHA file for the bank. But look at what it actually saves. From `src/app/admin/payroll/actions.ts`, in its own words:

> "*Save all the manually-typed lines for a run.*"

```
const net = dollarsToCents(String(formData.get(`net_${id}`) ?? ""));
```

The net pay is typed in by a human. This screen is a payment-distribution tool. It does not calculate anything — it takes the number you give it and moves that money. That is exactly what it was built for, and it is fine at that job.

Screen B — `/admin/books/net-pay`. This is the calculation engine, and it is genuinely excellent. It applies the correct order of operations, computes disposable earnings properly, applies garnishment ceilings, and explains every line in plain English. But its page reads:

```
const illustration = buildIllustrationScenario(ILLUSTRATION_DATE, "child_support");
const worked = buildNetPayWorkedExample(illustration.scenario);
```

It is a worked example. It computes a demonstration cheque from the rate registry to prove the engine is right. It never reads a real employee.

So: the screen that pays people cannot calculate, and the screen that calculates does not pay anybody. Neither is broken. They were built in that order deliberately — the engine had to be provably correct before it was allowed near real

money. But the seam between them has not been built, and until it is, you cannot run a payroll end to end.

What it takes: one slice — books-39. Everything it needs already exists:

Piece it needs	Does it exist?	Where
Hours for a pay period	Yes	`computeTimesheetForPeriod()`
Pay rates	Yes	`employee_pay` table (migration 0195)
Withholding calculation	Yes	`computeNetPay()` in `net-pay-core.ts`
Garnishment ceilings	Yes	`garnishment-core.ts`, 940 lines
Wage-base caps (SS, SUTA)	Yes	`loadYtdForEmployee()`, `applyRunToYtd()`
Tax rates for 2026	Yes	all 11 on file, verified
W-4 read-back	No — see 1.2	this is the missing piece

1.2 W-4 data is saved but never read back

This is the one genuinely missing component, and it is small but absolute.

W-4 forms are captured and stored in `employee_w4` (migration 0195). But every read of that table in the entire codebase looks like this:

```
admin.from("employee_w4").select("employee_id").in("employee_id", ids).eq("is_current", true)
```

It selects `employee_id` and nothing else. The system asks "does this person have a W-4 on file?" — a checkbox for the onboarding progress screen — and never asks "what does it *say*?"

The engine cannot compute federal income tax without the filing status, the step-2 checkbox, dependants, other income, deductions and extra withholding. Those values are sitting in the database right now, unread.

What it takes: one store function returning the current W-4 for an employee, plus the refusal path for when there is not one. Small, and it unlocks everything above it.

1.3 The garnishment lifecycle actions have no buttons

I found this while writing this report, and it is a fair example of the kind of gap worth catching.

`src/app/admin/books/garnishments/actions.ts` exports three working, tested server actions: `terminateWageOrderAction`, `suspendWageOrderAction`, `resumeWageOrderAction`. A search of every page and component for callers returns nothing.

They are correct, they are guarded, they are covered by tests — and there is no way to click them. So today you can enter an order and you cannot mark one finished. That matters in a specific way: a creditor writ expires sixty days after service, and when it does, somebody has to record that it is over. Right now that has to be done by me at the database, which is not acceptable for something with a legal deadline on it.

What it takes: buttons on the garnishment board, and a confirmation step on terminate. Small. It should ride along with books-39.

PART TWO — Tax forms and filings

2.1 There is no W-2, W-3, 941, or 940

I searched for any module producing these. There is none. `known-good-quarters.ts` and `payroll-reconciliation-report-core.ts` handle reconciliation, not form production.

To be clear about the boundary you and I agreed on: we are not becoming a filing agent. We replace the data-preparation half of what Aatrix does. So what is needed is the numbers, correct, in the boxes, in a form you can check and hand off — not electronic transmission to the IRS.

What this means concretely for your first year:

- Form 941, quarterly. First one due 30 April 2027 for Q1 2027.
- Form 940, annual FUTA. Due 31 January 2028 for 2027.
- W-2 and W-3, annual. Due 31 January 2028, and that is the deadline for *both* the copies to employees and the filing to the Social Security Administration.
- WA ESD quarterly report (account 000-073905-00-0).
- WA L&I quarterly report (account 521,756-00, risk class 6403).

The 941 is the urgent one by date, but it is also the easiest, because it is almost entirely a summation of things the YTD accumulator will already hold once payroll runs.

Sequencing note: none of these can be built properly until real pay runs exist to aggregate, because the only honest way to build them is against real numbers we can reconcile back to individual cheques. This is correctly *after* books-39, not before.

2.2 2027 rates: 10 of 11 are missing, and I cannot invent them

I probed the rate registry directly for this report:

```
2026-12-18   canRun: true    missing: 0 of 11
2027-01-01   canRun: false   missing: 10 of 11
```

The ten missing for 2027:

1. WA Paid Leave total premium rate
2. WA Paid Leave employee share
3. WA Paid Leave employer share
4. WA unemployment (SUTA) tax rate — *your* experience rate, specific to Greenway
5. WA unemployment wage base
6. L&I employee hourly rate — risk class 6403
7. L&I employer hourly rate — risk class 6403
8. Social Security wage base
9. Washington minimum wage
10. Federal minimum wage

The system refuses to compute a 2027 paycheque rather than carrying forward the 2026 figure. That refusal is deliberate and I want to defend it, because it will be inconvenient in December. A payroll system that silently reuses last year's SUTA rate produces a paycheque that is wrong by a small, plausible amount for every employee for the whole year, and nothing anywhere flags it. You find out when the agency does. A refusal is loud and takes an afternoon to clear; a silent wrong rate takes a year and an assessment notice.

You said the 2027 figures are a few weeks out. When the notices arrive, each one is entered with the document it came from attached as evidence.

PART THREE — Accounting beyond payroll

3.1 Financial statements exist as an engine with no screen

`financial-statements-core.ts` exists. `financial-statement-authorities.ts` exists. A search for anything in `src/app` or `src/components` importing them returns nothing.

There is a trial balance screen. There is no income statement, no balance sheet, no statement of cash flows, no statement of shareholder equity.

For an S-corp this is not cosmetic. You need a balance sheet for Schedule L, and you need retained earnings and distributions tracked properly for Schedule M-2 and the AAA. The engine for that (`basis-aaa-core.ts`) also exists, also unreachable.

3.2 Period close is built and not wired

`period-close-core.ts` and `period-close-authorities.ts` exist. No screen. So there is currently no way to lock a closed month

against back-dated entries — which is the control that stops last quarter's numbers changing after you have reported them.

3.3 Fixed assets and depreciation

`fixed-assets-core.ts` exists. No screen, and — more to the point — no depreciation schedule from you, which I have listed below under things only you can supply.

PART FOUR — 3,575 lines of guidance nobody can read

This is the largest single piece of finished, tested, unreachable work in the system, and I am listing it plainly because it represents real value sitting in the dark.

Nine mentor modules, all written, all tested, all with zero imports from any page or component:

Module	Lines	Teaches
`payroll-onboarding-mentor`	746	Hiring paperwork, W-4, I-9
`tax-penalty-mentor`	540	Which penalty applies and how to abate it
`financial-statements-mentor`	400	Reading your own statements
`cogs-position-mentor`	371	280E and cost of goods sold — material to a cannabis retailer
`basis-aaa-mentor`	368	S-corp basis and the accumulated adjustments account
`interest-mentor`	328	Interest computation and the federal rates
`internal-control-mentor`	316	Separation of duties in a small shop
`period-close-mentor`	281	Closing a month properly
`s-corporation-year-mentor`	226	The S-corp annual cycle
Total	3,575	

This is standing rule 50 at scale — code that passes its tests and cannot be reached by a human being. Each one needs a screen, or a panel on an existing screen. The COGS/280E one is the one I would wire first: it is the single largest tax issue a Washington cannabis retailer has, and 371 lines of written guidance on it are currently invisible.

The same is true of two calculation engines, not just the guidance

The nine modules above teach. These two *compute*, and they are equally unreachable:

Module	Lines	Computes	Blocks
`financial-statements-core`	1,600	Income statement, balance sheet, cash flows, equity	Schedule L, Schedule M-2
`period-close-core`	868	Month/quarter lock against back-dated entries	The control that keeps a reported quarter from changing after you report it

I am naming them here with the same specificity as the mentor modules because a claim like "the accounting side needs work" is not actionable, and because these two are the difference between a bookkeeping tool and a system that can produce a defensible year end.

A note on how this document is kept honest. Every claim in this part — that these eleven modules exist, and that nothing in `src/app` or `src/components` imports them — is re-derived by an automated check that runs on every commit (`tests/compliance/owner-report-books-38.test.ts`). If someone builds one of these screens, that check fails, and the failure message tells them to update this report. That is deliberate. A gap report that quietly goes stale is worse than no gap report, because you would plan around it. So the day one of these lines becomes untrue is the day the build tells us to delete it.

PART FIVE — Engineering debt I am carrying

Recorded honestly, because you should know what I know.

Eleven copies of the same regular expression. Eleven separate test files each define their own private `/^export function/gm` scan to enumerate a module's exports. There is a shared helper, `exportedFunctionNames`, that eleven files should be using and are not. When the pattern needs to change, it needs changing in eleven places, and the eleventh will be missed.

29 U.S.C. § 206(a)(1) is not mirrored. The federal minimum wage statute is cited in the code but its verbatim text is not stored alongside the others in an authorities file. Every other statute we rely on is mirrored with a test that fails if the quote is altered. This one is an exception and should not be.

No end-to-end browser test. Everything is verified at the source and unit level, which catches a great deal — this very slice caught three real defects that way. What it does not catch is a form that renders but cannot actually be submitted by a human. The three defects I found in phase F were all found by *reading*, not by a test, which tells me the source-level gates have a ceiling.

`next build` cannot run in my sandbox — it is killed for memory. I verify with the TypeScript compiler and rely on CI for the real build. It has not bitten us, but it means the build is the one thing I cannot check before pushing.

PART SIX — What only you can supply

I will not guess at any of these, and each one blocks something specific.

For the 2027 cutover:

1. 2027 WA minimum wage (L&I announces ~September)
2. 2027 L&I rate notice — your risk class 6403 rates
3. 2027 ESD SUTA rate notice — your experience rate
4. 2027 PFML premium split
5. 2027 SSA wage base

For the S-corp return: 6. The S-election tax year, and a copy of the accepted election 7. Ending AAA from the most recent Schedule M-2 8. Any §6699(e) amount assessed 9. Quarterly federal short-term rates for the interest engine 10. The depreciation schedule for existing fixed assets

For the conversion from Sage: 11. Work papers supporting the current trial balance 12. The twelve years of filed returns 13. Intercompany detail, if there is any 14. Wells Fargo loan details — original amount, rate, term, current balance

---

PART SEVEN — What is actually done, and done well

A gap report that lists only gaps is misleading. Here is the other side, all verified:

- **Withholding engine.** Federal income tax via percentage method, Social Security with the wage-base cap, Medicare including Additional Medicare, WA Paid Leave, WA Cares, L&I. Correct order of operations. 2,185 lines, heavily tested.
- **Garnishment engine.** All seven order kinds, correct ceilings, competing-order priority, the twelve-week and second-family adjustments. 940 lines, mutation-tested against the statutes.
- **Garnishment entry.** As of this slice, complete — with per-field teaching, refusals that explain themselves, and 24 mutation-proved gates.
- **Timesheets and overtime.** Workweek settings, pay periods, approval flow.
- **Sick leave.** Washington accrual rules, the leave inbox, the generosity board.
- **Year-to-date accumulators.** Per employee, per tax, with apply and unapply.
- **Rate registry with evidence.** Every rate carries the notice it came from, and the system refuses to compute without one.
- **Verbatim statutes.** Every authority we depend on is stored word for word with a test that fails if the text is altered.
- **Banking and ACH.** NACHA generation, encrypted at rest, guardrails before release, and a payee vault so a paycheque cannot be redirected from the payroll screen.
- **8,942 tests across 403 files**, and — the part I care about more — every gate proved failable by deliberately breaking the thing it guards.

PART EIGHT — The order I would build in

Next — books-39: the pay run. W-4 read-back, then wire timesheet → engine → YTD → pay run for a real employee, on a real date. Add the garnishment lifecycle buttons while I am there. **This is what unlocks your parallel run against Sage on 2026 data.**

Then — books-40: quarterly filings. 941 first, since it is due first and is mostly summation; then the ESD and L&I quarterly reports.

Then — books-41: financial statements. Income statement, balance sheet, equity — wiring an engine that already exists. Period close alongside it.

Then — books-42: annual forms. W-2, W-3, 940. These need a full year of runs behind them to be built honestly.

Throughout — wiring the mentor modules. COGS/280E first.

When your notices arrive — 2027 rates. An afternoon each, with the notice attached as evidence.

Every factual claim in this document was verified against the code on the date of writing. Where I found something missing, I ran the search. Where I found something built but unreachable, I checked both ends. Nothing here is from memory.